

Лабораторна робота №1

ПРИХОВУВАННЯ ДАНИХ У ЗОБРАЖЕННІ МЕТОДОМ ЗАМІНИ НАЙМЕНШОГО ЗНАЧУЩОГО БІТА (LSB)

Мета: дослідити принципи роботи методу LSB на основі навчальної програми СгупTool 2; проаналізувати ємність стеганографічного приховування даних; реалізувати алгоритм LSB для приховування текстових даних у зображенні, а також функціонал для вилучення прихованого тексту з зображення.

Завдання 1. Виконати приховування повідомлення у зображенні методом LSB на основі навчальної програми СгупTool 2.

2. Відкрийте проєкт Image Steganography with LSB, введіть своє ім'я та прізвище у поле для секретного повідомлення. Також, у відповідному полі, оберіть зображення, що буде слугувати контейнером

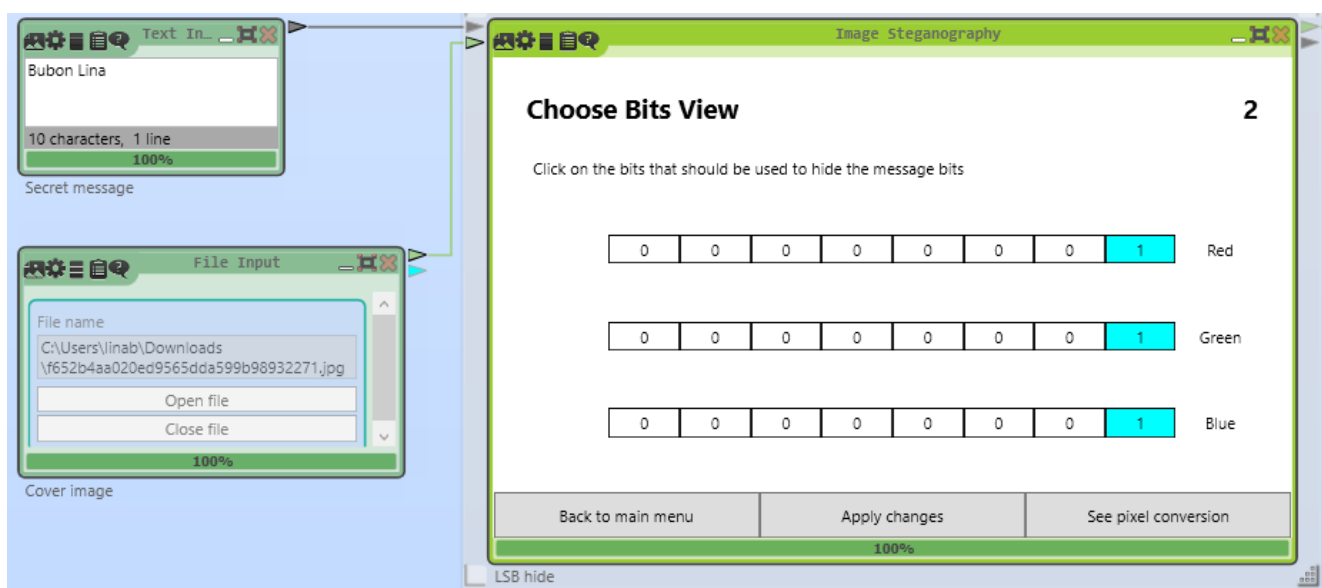


Рис. 1. Введення повідомлення.

					ЖИТОМИРСЬКА ПОЛІТЕХНІКА.26.121.03.000 – Лр.1						
Змн.	Арк.	№ докум.	Підпис	Дата	Звіт з лабораторної роботи №1			Літ.	Арк.	Аркушів	
Розроб.		Бубон Л. В.									
Перевір.		Кириєнко М. Ю.								1	8
Реценз.								ФІКТ, гр. ІПЗ-23-5			
Н. Контр.											
Зав.каф.											

3. Запустіть програму на виконання (натисніть Play у головному меню програми) та оберіть кількість бітів для модифікації за допомогою опції Choose bits for hiding process. Оберіть один найменш значущий біт та настаніть Apply changes. Після цього дані вбудуються у контейнер та згенерується нове зображення зі стеганографічними даними.

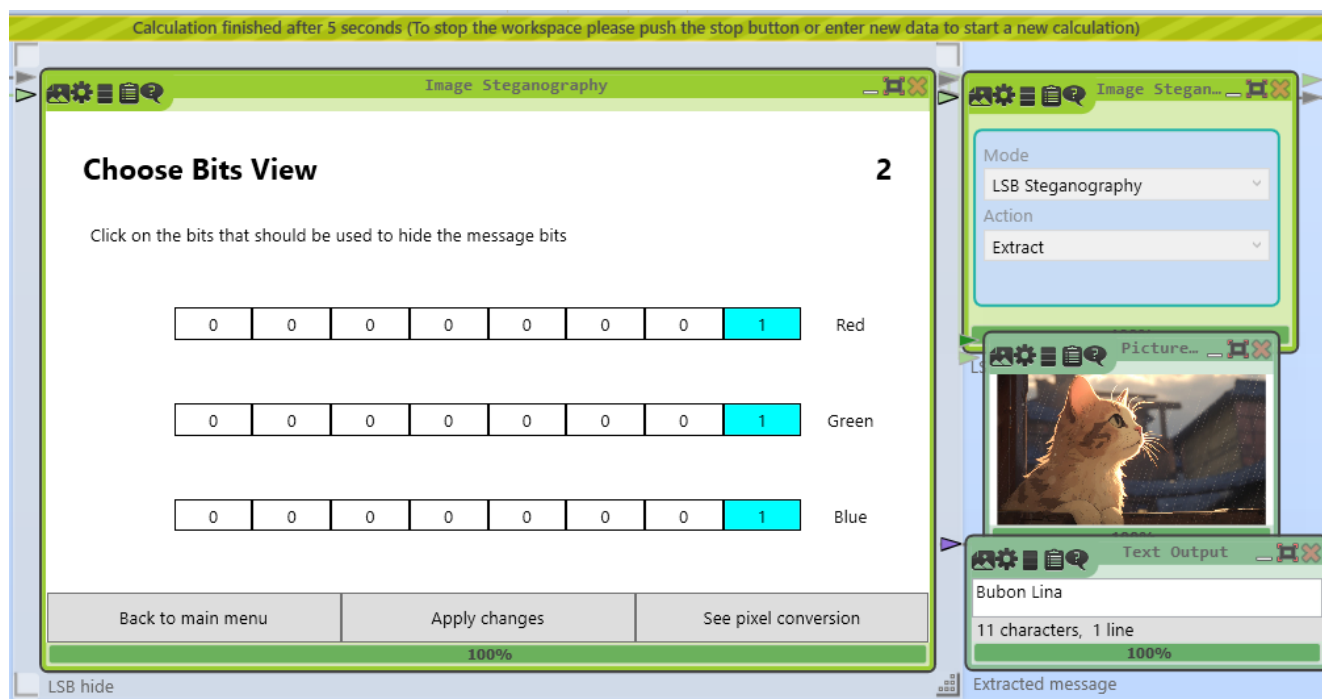


Рис. 2. Запуск та результат виконання.

4. Натисніть опцію See pixel conversation та перегляньте процес заміни пікселів. Детально опишіть у звіті приховування методом LSB першої та останньої літери секретного повідомлення.

Перша літера

Остання літера

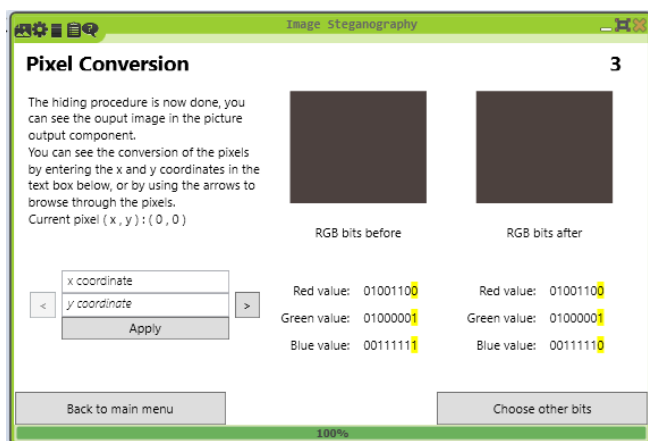


Рис. 3. Модифікація пікселя (0, 0)

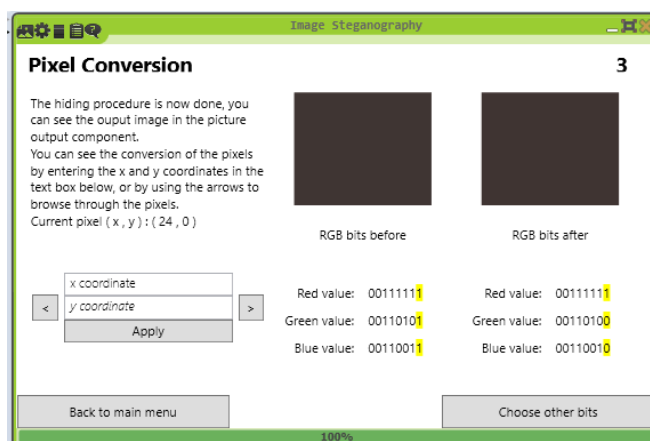


Рис. 4. Модифікація пікселя (24, 0)

Перша літера

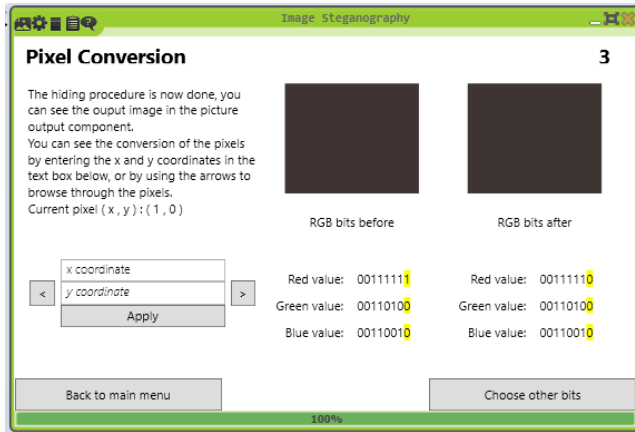


Рис. 5. Модифікація пікселя (1, 0)

Остання літера

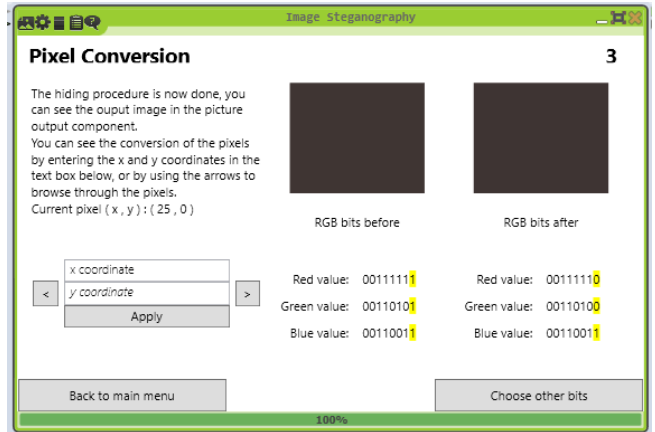


Рис. 6. Модифікація пікселя (25, 0)

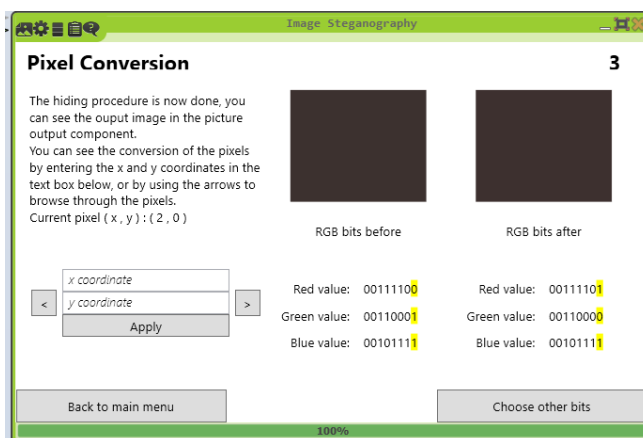


Рис. 7. Модифікація пікселя (2, 0)

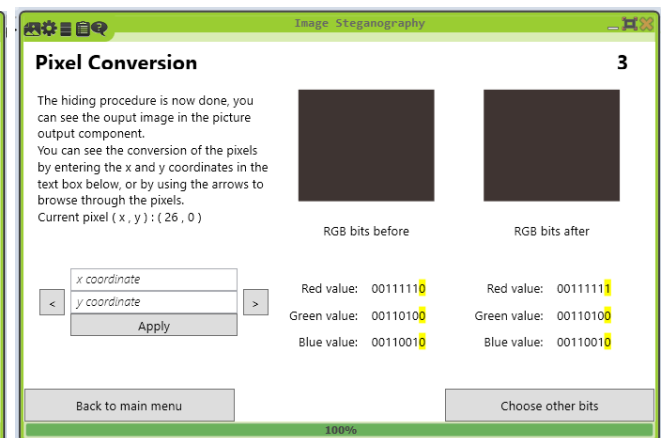


Рис. 8. Модифікація пікселя (26, 0)

5. За допомогою опції Overview and Hiding Capacity Information перегляньте інформацію про довжину повідомлення та ємність приховування. Самостійно обчисліть довжину повідомлення та максимальну ємність приховування, якщо замінювати 1 найменш значущий біт. Порівняйте власноруч обчислені значення із тими, що на скриншоті.

Повідомлення для шифрування: Bubon Lina

К-сть символів з пробілом 10

Загальна довжина повідомлення $10 * 8 = 80$ бітів

Зображення розміром 2141x1200

Максимальна ємність приховування $C = 2141 * 1200 * 3 * 1 = 7\,707\,600$ бітів

					ЖИТОМИРСЬКА ПОЛІТЕХНІКА.26.121.03.000 – Лр.1	Арк.
Змн.	Арк.	№ докум.	Підпис	Дата		3

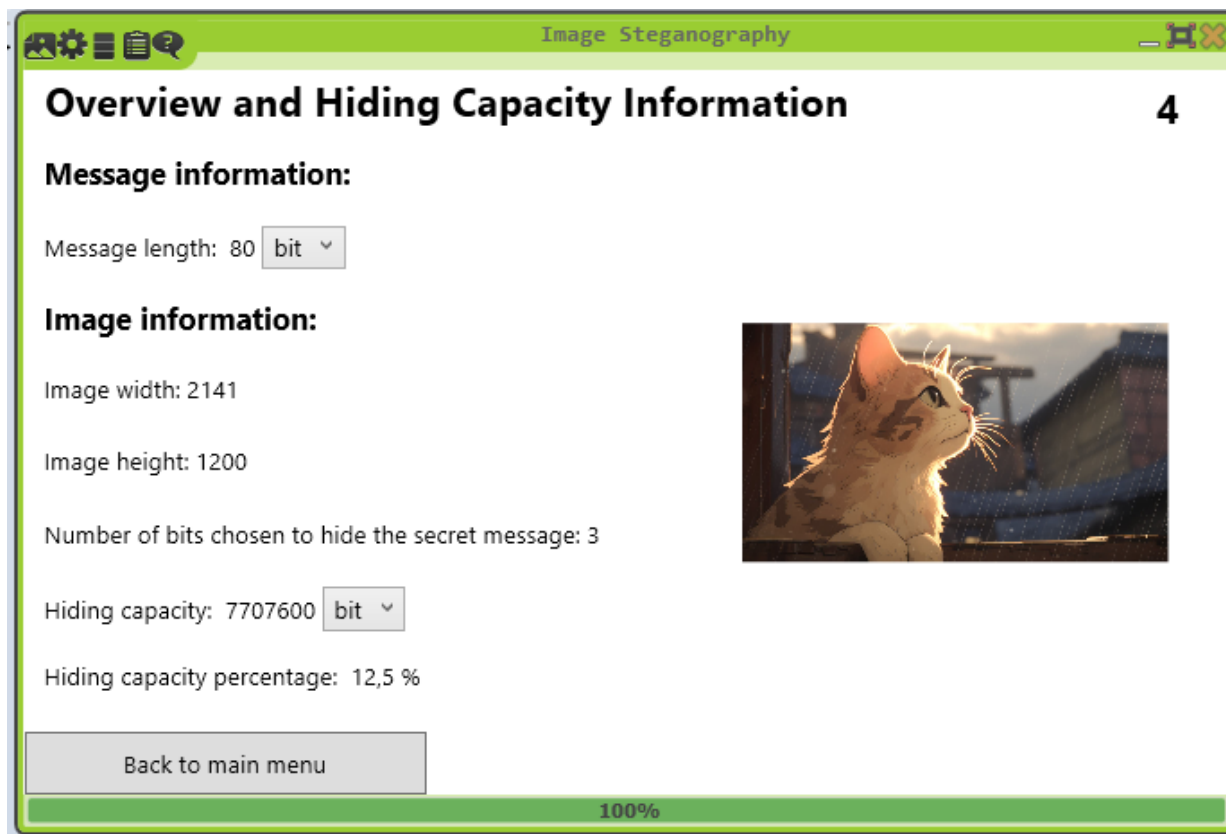


Рис. 9. Інформація про довжину повідомлення та ємність приховування

6. Забезпечте збереження стегоконтейнеру до нового файлу. Для цього додайте до проєкту компонент File Output, використовуючи розділ Tools на панелі модульних компонентів Components. Ця панель розміщена ліворуч робочого вікна та дає змогу додавати до проєкту нові складові, таким чином удосконалюючи його.



Рис. 10. Додавання до проєкту компоненту File Output

7. Знову запустіть програму на виконання. Порівняйте у звіті оригінальне зображення та зображення із прихованими даними



Рис 11. Оригінальне фото



Рис. 12. Фото з прихованим текстом

9. Створіть новий проект для вилучення прихованого повідомлення, наприклад під назвою LSB_dec.cwm. Додайте до проекту компоненти File Input, Image Steganography та Text Output. Встановіть зв'язки, налаштуйте параметри роботи складових проекту.

10. Збережіть проект та перевірте його роботу.

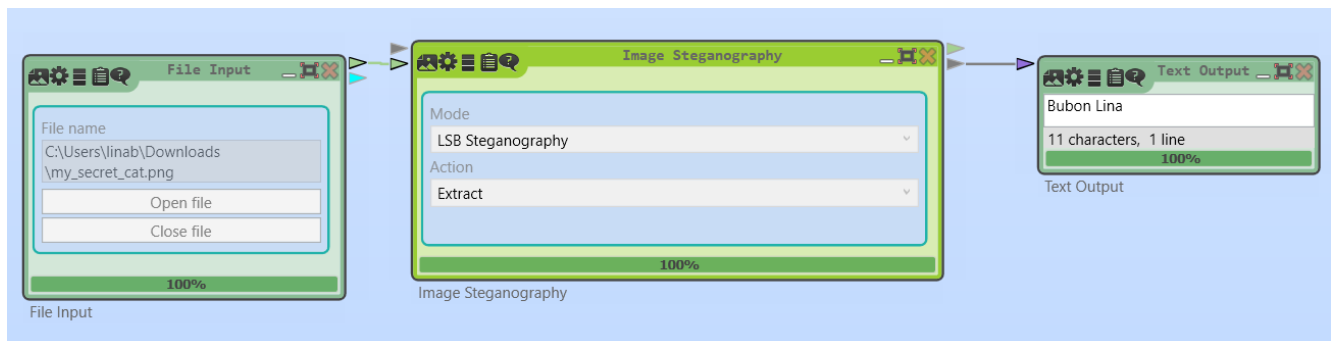


Рис. 16. Вивід тексту.

8. Обміняйтеся зображенням із вбудованими даними. Додайте отримане зображення до звіту.

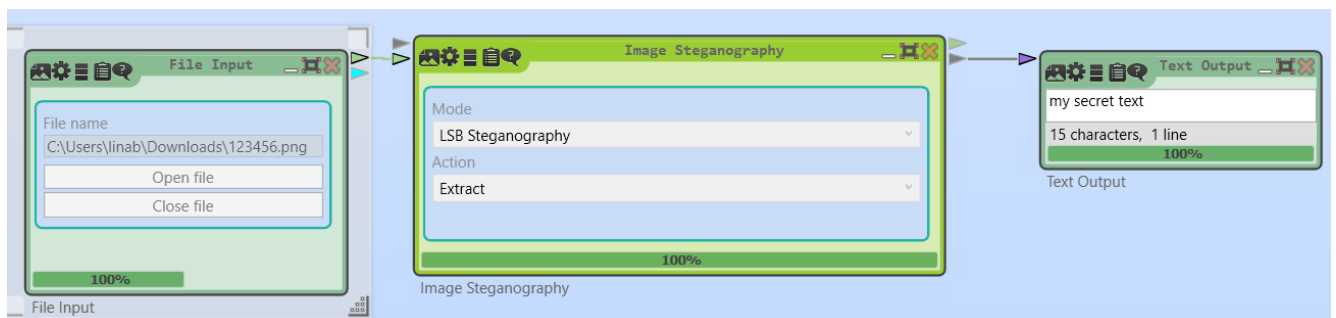


Рис. 13. Вивід тексту з картинки.

					ЖИТОМИРСЬКА ПОЛІТЕХНІКА.26.121.03.000 – Лр.1	Арк.
Змн.	Арк.	№ докум.	Підпис	Дата		5



Рис 14. Оригінальне фото



Рис. 15. Фото з прихованим текстом

Завдання 2. Розробити програму мовою програмування для приховування текстових даних у зображенні методом LSB. Реалізувати функціонал для вилучення прихованого тексту із зображення. Виконати порівняння оригінального зображення та зображення із прихованими даними. Реалізувати інтерфейс взаємодії з користувачем – пропонувати вибір між приховуванням та вилученням тексту, запитувати текст для приховування та шлях до зображення. Забезпечити роботу із зображеннями у форматі PNG або BMP. Перевірити коректність роботи програми на різних зображеннях.

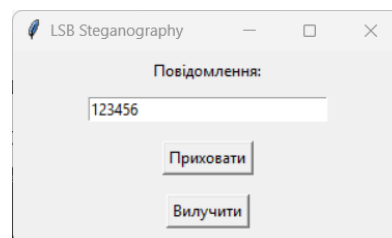


Рис. 16. Приховування повідомлення.

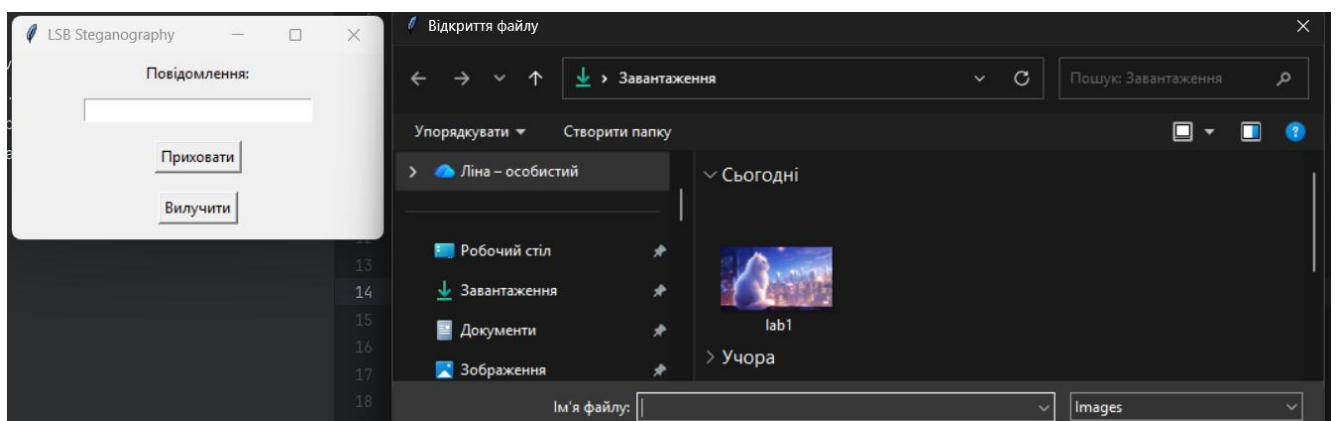


Рис. 17. Обрання шляху.

					ЖИТОМИРСЬКА ПОЛІТЕХНІКА.26.121.03.000 – Лр.1	Арк.
Змн.	Арк.	№ докум.	Підпис	Дата		6

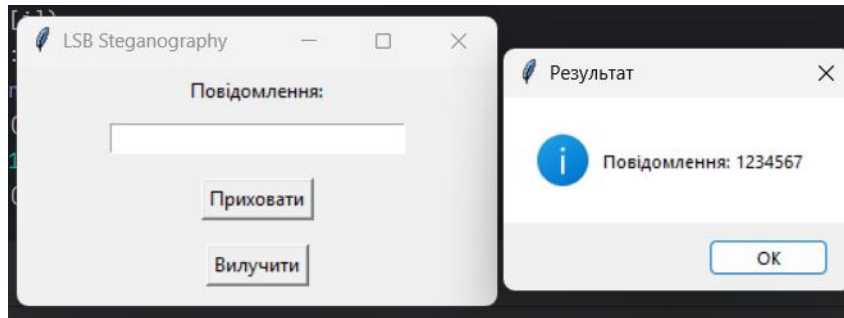


Рис. 18. Вилучення повідомлення



Рис 19. Оригінальне фото



Рис. 20. Фото з прихованим текстом

Лістинг програми:

```
import tkinter as tk
from tkinter import filedialog, messagebox
from PIL import Image

def encode_img(img_path, text, out_path):
    text += "END"
    bin_msg = ''.join(format(ord(c), '08b') for c in text)
    img = Image.open(img_path)
    pixels = list(img.getdata())
    if len(bin_msg) > len(pixels) * 3:
        raise ValueError("Повідомлення занадто велике для цієї картинки.")
    b_idx = 0
    for i in range(len(pixels)):
        pix = list(pixels[i])
        for j in range(3):
            if b_idx < len(bin_msg):
                pix[j] = (pix[j] & ~1) | int(bin_msg[b_idx])
                b_idx += 1
        pixels[i] = tuple(pix)
    new_img = Image.new(img.mode, img.size)
    new_img.putdata(pixels)
    new_img.save(out_path)

def decode_img(img_path):
    img = Image.open(img_path)
    pixels = list(img.getdata())
    bin_msg = ""

    for pix in pixels:
        for val in pix[:3]:
            bin_msg += str(val & 1)
```

					ЖИТОМИРСЬКА ПОЛІТЕХНІКА.26.121.03.000 – Лр.1	Арк.
						7
Змн.	Арк.	№ докум.	Підпис	Дата		

```

        res_text = ""
    for i in range(0, len(bin_msg), 8):
        byte = bin_msg[i:i + 8]
        char = chr(int(byte, 2))
        res_text += char
        if res_text[-3:] == "END":
            return res_text[:-3]

    raise ValueError("Приховане повідомлення не знайдено.")

def action_hide():
    path_in = filedialog.askopenfilename(filetypes=[("Images", "*.png *.bmp")])
    if not path_in:
        return
    msg = entry_msg.get()
    if not msg:
        messagebox.showerror("Помилка", "Введіть повідомлення")
        return
    path_out = filedialog.asksaveasfilename(defaultextension=".png",
    filetypes=[("PNG", "*.png")])
    if not path_out:
        return
    try:
        encode_img(path_in, msg, path_out)
        messagebox.showinfo("Успіх", "Приховано успішно")
    except Exception as e:
        messagebox.showerror("Помилка", str(e))

def action_extract():
    path_in = filedialog.askopenfilename(filetypes=[("Images", "*.png *.bmp")])
    if not path_in:
        return
    try:
        res = decode_img(path_in)
        messagebox.showinfo("Результат", f"Повідомлення: {res}")
    except Exception as e:
        messagebox.showerror("Помилка", str(e))

root = tk.Tk()
root.title("LSB Steganography")
root.geometry("300x150")
label_msg = tk.Label(root, text="Повідомлення:")
label_msg.pack(pady=5)
entry_msg = tk.Entry(root, width=30)
entry_msg.pack(pady=5)
btn_hide = tk.Button(root, text="Приховати", command=action_hide)
btn_hide.pack(pady=10)
btn_extract = tk.Button(root, text="Вилучити", command=action_extract)
btn_extract.pack(pady=5)
root.mainloop()

```

Висновок: у ході лабораторної роботи №1 я дослідила принципи роботи методу LSB на основі навчальної програми CrypTool 2, а також проаналізувала ємність стеганографічного приховування даних, та реалізувала алгоритм LSB для приховування текстових даних у зображенні, а також функціонал для вилучення прихованого тексту з зображення.

					ЖИТОМИРСЬКА ПОЛІТЕХНІКА.26.121.03.000 – Лр.1	Арк.
						8
Змн.	Арк.	№ докум.	Підпис	Дата		